8.1 飞书钉钉企业微信集成

> **生成模型**：gpt-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~18k tokens，输出 ~4.5k tokens（估算，本节）

如果你在国内落地 OpenClaw，最先撞上的往往不是代码，而是平台。国外教程一上来就是 Telegram、Discord、Slack，可国内团队每天真正在用的，通常是飞书、钉钉、企业微信。你不把这几个平台接好，OpenClaw 再强，也很难进日常工作流。

这一节不讲扩展源码怎么写，重点只讲一件事：作为使用者，怎么把 OpenClaw 接进国内协作平台，并让它真的能收消息、回消息、稳定跑起来。

8.1.1 为什么这几个平台通常做成扩展

OpenClaw 里的通道大体分两类：

1. **核心通道**：主仓库长期维护，和 Gateway 配合更紧，文档也更稳定。
2. **扩展通道**：通过插件或扩展包挂进去，核心 Gateway 不改，按需增加平台适配能力。

飞书、钉钉、企业微信这类国内平台，很多时候更适合放在扩展层。原因不复杂：

- 企业平台 API、权限模型、审核规则变化快。
- 同一个平台里，经常同时存在“内部应用”“群机器人”“Webhook 回调”“事件订阅”几种路线。
- 不同公司的安全要求差异很大，做成扩展更灵活。

你可以把扩展理解成 Gateway 旁边的一层翻译器。它负责三件事：

- 把平台的入站事件转成 OpenClaw 能理解的标准消息。
- 把 OpenClaw 的回复转成平台 API 能接受的格式。
- 处理平台自己的认证、签名、限流、回调校验。

也正因为如此，做中国平台接入时，真正的工作量经常不在 Agent 逻辑里，而在“证书、域名、权限、回调、加签”这些外围条件上。

8.1.2 开始前先准备这些东西

建议一次性准备好下面这些，不然会不断卡住：

- 一台公网可访问的服务器，最好在中国大陆云厂商上。
- 一个域名，正式环境最好已经完成 ICP 备案。
- 一个能签 HTTPS 证书的入口域名。
- 一套 OpenClaw 基础配置文件。
- 一套环境变量文件，不要把密钥硬写进配置。
- 一个在国内可稳定访问的模型提供者，比如通义、DeepSeek、智谱，或者你自己做好的代理。

目录可以先按这个方式摆：

```
~/openclaw/
├── openclaw.json
├── .env
├── logs/
└── data/
```

`.env` 示例：

```
export FEISHU_APP_SECRET="replace-me"
export FEISHU_VERIFICATION_TOKEN="replace-me"
export FEISHU_ENCRYPT_KEY="replace-me"
export DINGTALK_CLIENT_SECRET="replace-me"
export DINGTALK_ROBOT_SECRET="replace-me"
export DINGTALK_BRIDGE_SECRET="replace-me"
export WECOM_AGENT_SECRET="replace-me"
export WECOM_TOKEN="replace-me"
export WECOM_ENCODING_AES_KEY="replace-me"
export OPENCLAW_GATEWAY_TOKEN="replace-me"
```

加载环境变量：

```
set -a
source ./.env
set +a
```

如果服务器在中国大陆，顺手把镜像源也切了，后面装依赖会舒服很多：

```
npm config set registry https://registry.npmmirror.com
pnpm config set registry https://registry.npmmirror.com
```

8.1.3 飞书集成：最适合先打通的一条线

飞书通常是这三个里最好上手的。文档相对完整，机器人和事件订阅思路也比较清楚。如果你要在国内公司里第一次试点 OpenClaw，我通常建议先从飞书开始。

第一步：创建企业自建应用

在飞书开放平台里，一般按这个顺序走：

1. 创建企业自建应用。
2. 开启机器人能力。
3. 配置消息相关权限。
4. 记录 `App ID` 和 `App Secret` 。
5. 如果走事件订阅，再配置 `Verification Token` 、 `Encrypt Key` 和回调地址。

飞书常见有两种接法：

- **长连接模式**：OpenClaw 主动连平台，调试时最省事。
- **Webhook 模式**：飞书把事件推给你的服务，更适合正式环境。

第一次接，建议先长连接验证业务，再切 Webhook。因为先把“能聊天”打通，比一上来就折腾域名、备案、HTTPS、反向代理更重要。

第二步：OpenClaw 配置

不同扩展的字段名可能略有差别，但大体长这样：

```
{
  gateway: {
    token: "$OPENCLAW_GATEWAY_TOKEN"
  },
  agents: {
    main: {
      model: "qwen-max",
      systemPrompt: "你是部署在飞书里的企业助手。"
    }
  },
  channels: {
    feishu: {
      enabled: true,
      dmPolicy: "pairing",
      mode: "webhook",
      accounts: {
        main: {
          appId: "cli_xxxxxxxxx",
          appSecret: "$FEISHU_APP_SECRET",
          verificationToken: "$FEISHU_VERIFICATION_TOKEN",
          encryptKey: "$FEISHU_ENCRYPT_KEY",
          endpoint: "https://bot.example.cn/hooks/feishu",
          botName: "OpenClaw 助手"
        }
      }
    }
  }
}
```

如果你先走长连接，一般就是把 `mode` 改成扩展要求的长连接模式，并临时去掉 `endpoint` 、 `verificationToken` 这种 Webhook 专用字段。

第三步：平台侧权限和事件订阅

飞书那边至少要确认这几件事：

- 允许机器人接收私聊消息。
- 允许机器人在群里被 @ 时接收消息。
- 允许机器人发消息。
- 如果做事件订阅，回调 URL 已经填好并通过校验。

这里有个很常见的坑：你以为“能发消息”就够了，实际上私聊权限、群聊权限、事件订阅权限经常是拆开的。结果就是机器人看起来在线，但一 @ 它就没反应。

第四步：启动和测试

```
openclaw gateway --config ./openclaw.json --allow-unconfigured
```

看日志：

```
openclaw logs --follow
```

测试顺序建议这样走：

1. 先用管理员账号给机器人发私聊。
2. 再把机器人拉进测试群，手动 @ 它。
3. 看 OpenClaw 日志里有没有入站事件。
4. 看消息有没有成功回发。

如果走的是 Webhook，再补一刀 URL 连通性测试：

```
curl -i https://bot.example.cn/hooks/feishu
```

飞书这条线的限制

- 企业管理员审批链可能很长，权限不一定你自己说开就开。
- 有的公司会要求域名备案主体必须和企业主体一致。
- 飞书消息进来了，不代表模型侧就一定稳定。如果你的模型服务在境外，回复可能很慢。
- 群聊里消息格式经常比私聊复杂，尤其是富文本、卡片消息和附件。

8.1.4 钉钉集成：更适合“机器人 + 中间层”打法

钉钉也能接，但实战里通常比飞书更绕。你会同时碰到这些概念：企业内部应用、群机器人、自定义机器人、回调 URL、安全加签。第一次接的时候，最容易懵的就是不知道自己到底该走哪条线。

如果目标只是“先让 OpenClaw 在群里或者内部应用里回消息”，更推荐这个路线：

- 群场景先走 **钉钉机器人**。
- 需要企业身份、组织权限时，再上 **内部应用**。

- 如果现成扩展不完整，就加一层 **bridge 服务**，别让 OpenClaw 直接和钉钉所有细节耦死。

第一步：创建机器人或内部应用

创建完成后，你通常会拿到这些信息：

- `Client ID`
- `Client Secret`
- 机器人 `Webhook URL`
- 机器人签名 `Secret`

钉钉的一个现实问题是：发消息这边经常能直接用机器人 Webhook，收消息却不一定能直接映射到 OpenClaw，所以很多团队最后都会多加一层桥接服务。

第二步：推荐的接入结构

一个很实用的结构是：

```
钉钉消息 -> 钉钉回调或事件订阅 -> 你的 bridge 服务 -> OpenClaw Gateway/Webhook
OpenClaw 回复 -> bridge 服务 -> 钉钉发送接口或机器人 Webhook
```

这样做有几个好处：

- 平台签名、时间戳校验都留在 bridge 里处理。
- OpenClaw 侧只收统一格式，后面换平台时不用重做一遍 Agent 配置。
- 有些企业会把平台接入服务和业务服务分开部署，这种结构也更方便审计。

第三步：OpenClaw 示例配置

下面这个配置是“通过扩展 + bridge”方式的示例。字段名可能会随扩展实现变化，但结构基本就这个意思：

```jsonc
{
  channels: {
    dingtalk: {
      enabled: true,
      dmPolicy: "pairing",
      transport: "bridge",
      accounts: {
        main: {
          clientId: "dingxxxxxxxx",
          clientSecret: "$DINGTALK_CLIENT_SECRET",
          robotWebhook: "https://oapi.dingtalk.com/robot/send?access_token=xxxxx",
          robotSecret: "$DINGTALK_ROBOT_SECRET",
          callbackEndpoint: "https://bot.example.cn/hooks/dingtalk",
          bridgeSharedSecret: "$DINGTALK_BRIDGE_SECRET"
        }
      }
    }
  }
}
```

如果你的 bridge 服务需要把消息转给 OpenClaw，可以先做一个最简单的转发：

```bash
curl -X POST https://127.0.0.1:8945/webhook/dingtalk \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer replace-me' \
  -d '{
    "channel": "dingtalk",
    "sender": "u123456",
    "text": "@OpenClaw 帮我总结今天的日报"
  }'
```

第四步：测试重点

钉钉这边不要一上来就测复杂场景，按这个顺序来：

1. 先测机器人是否能主动发消息。
2. 再测消息能不能从钉钉进到 bridge。
3. 再测 bridge 能不能转给 OpenClaw。
4. 最后再测 OpenClaw 回复能否回到原会话。

这一步很关键。因为钉钉接入失败时，问题可能出在四层里任意一层。你如果不拆开测，日志看起来就只剩一句“没收到消息”，排查会非常痛苦。

钉钉这条线的限制

- 平台能力经常按企业版本、管理员权限、应用类型区分。
- 机器人能做的事，未必等于内部应用能做的事。
- Webhook 和事件回调的安全校验比较多，时间戳和签名很容易配错。
- 有些公司对外部回调域名限制很严，必须白名单或者走专线。

8.1.5 企业微信集成：最像“企业内部系统接入”

企业微信的气质和前两个不太一样。它不是单纯一个聊天工具，更像公司内部账号体系、审批和通讯录能力的入口。所以它很适合做“公司内部助手”，但前提是你得把应用和企业权限关系理顺。

第一步：创建企业微信自建应用

通常流程是：

1. 企业管理员进入企业微信后台。
2. 创建自建应用。
3. 获取 `CorpID` 、 `AgentID` 、 `Secret` 。
4. 如果做消息接收，配置 `Token` 、 `EncodingAESKey` 、回调 URL。
5. 把应用可见范围授给测试部门或测试人员。

企业微信最容易忽略的一点，是“可见范围”。你以为应用建好了，实际上只有管理员自己能看到，普通同事根本找不到这个机器人。

第二步：OpenClaw 示例配置

```
{
  channels: {
    wecom: {
      enabled: true,
      dmPolicy: "pairing",
      mode: "callback",
      accounts: {
        main: {
          corpId: "wwxxxxxxxxxxxx",
          agentId: "1000002",
          secret: "$WECOM_AGENT_SECRET",
          token: "$WECOM_TOKEN",
          encodingAESKey: "$WECOM_ENCODING_AES_KEY",
          endpoint: "https://bot.example.cn/hooks/wecom"
        }
      }
    }
  }
}
```

如果你的扩展实现支持 HTTP 中转，也可以像钉钉一样把企业微信回调先交给 bridge，再统一转入 OpenClaw。

第三步：平台侧消息回调和可信 IP

企业微信回调配置时，经常要确认这些：

- 回调地址必须是公网 HTTPS。
- `Token` 和 `EncodingAESKey` 要和 OpenClaw 或 bridge 配置一致。
- 某些企业安全策略下，还会要求配置可信 IP。
- 应用必须对目标员工或部门可见。

可信 IP 这件事对新手特别容易造成错觉：你本地调试明明没问题，一上云就不通，结果最后发现企业微信后台限制了来源地址。

第四步：测试命令和测试顺序

启动 OpenClaw：

```
openclaw gateway --config ./openclaw.json --allow-unconfigured
```

检查回调入口：

```
curl -i https://bot.example.cn/hooks/wecom
```

然后按这个顺序测：

1. 管理员账号先发一条私聊。
2. 测试部门成员再发一条私聊。
3. 把应用拉进测试群。
4. 发一条 @ 机器人消息。
5. 观察日志里会话 key 是否稳定。

企业微信这条线的限制

- 企业后台权限重，很多配置都得管理员配合。
- 可见范围和通讯录权限不理清，应用常常“存在但没人能用”。
- 回调加解密、签名校验比普通聊天机器人更偏企业系统风格。
- 如果你想让 OpenClaw 调组织架构、审批流、打卡之类接口，那已经不是简单聊天集成，而是内部系统接入了，复杂度会明显上去。

8.1.6 三个平台怎么选

如果你是第一次在中国团队里部署 OpenClaw，我的建议很直接：

- **想先跑通**：优先飞书。
- **公司主阵地是钉钉**：先机器人，后内部应用，再考虑 bridge 标准化。
- **要做公司内部助手**：优先企业微信，但要提前拉管理员一起配。

一个很实际的推荐顺序是：

```
# 1. 先确定模型能稳定跑
openclaw models set qwen-max

# 2. 再启动网关
openclaw gateway --config ./openclaw.json --allow-unconfigured

# 3. 然后只接一个平台做闭环验证
openclaw logs --follow
```

先把一条线彻底打通，比三条线同时半通不通强太多。

8.1.7 中国大陆环境下的几个现实问题

最后再说几个很现实的点，这些比代码更容易把项目拖住。

一是 GFW 和跨境链路

平台在国内，不代表你的模型、依赖源、镜像源也在国内。如果 OpenClaw 收到消息以后还要跨境访问模型，体验会明显抖动。对公司内部场景来说，用户不会在意你是不是“架构优雅”，他们只会觉得这个机器人怎么一会儿快一会儿慢。

所以更稳的做法是：

- 生产环境优先选国内可稳定访问的模型服务。
- npm、Docker、apt 尽量换国内镜像源。
- 平台回调和模型调用尽量不要同时跨境。

二是 ICP 备案

只要你要正式用 Webhook，又希望域名在中国大陆稳定可访问，ICP备案基本绕不过去。别等功能全做完才想这件事，不然很可能最后卡在“服务是好的，但域名不能正式上线”。

三是日志和审计

企业平台接入后，最怕的不是代码错，而是“没证据”。建议至少保留下面几类日志：

- 平台回调日志
- OpenClaw 入站事件日志
- 模型调用耗时日志
- 出站发送日志

这样出了问题，你能很快判断是平台没推、桥接没转、Gateway 没收，还是模型超时。

本节小结

中国平台集成的关键，不是把 OpenClaw 接到一个聊天工具上就算完，而是把“扩展层、平台权限、回调入口、国内网络环境”一起想清楚。飞书最适合先跑通；钉钉更适合机器人加中间层的打法；企业微信最像企业内部系统接入，管理员配合非常重要。把这三条线想明白了，后面做国内部署和生产化就顺多了。